

Clean-Code-Analyse — Adressverwaltung v2.0.0

Datum: September 2026
Geprüfte Dateien: Backend (C#) + Frontend (TypeScript/React)
Grundlage: Clean Code (Robert C. Martin), SOLID-Prinzipien, DRY

Bewertungsübersicht

Kategorie	Befunde	Priorität
SRP — Single Responsibility	3 Verletzungen	● Hoch
DRY — Don't Repeat Yourself	3 Verletzungen	● Mittel
Fehlerbehandlung	3 Probleme	● Hoch
Validierung	1 Problem	● Mittel
Konfiguration / Magic Strings	2 Probleme	● Mittel
TypeScript / Typsicherheit	2 Probleme	● Niedrig
Namensgebung	2 Probleme	● Niedrig
UI-Konsistenz	1 Problem	● Niedrig

Befunde — Backend (C#)

B-01 ● SRP: Seed-Code in Program.cs (Startup-Datei)

Datei: backend/AdressverwaltungApi/Program.cs
Zeilen: ~80–95

Problem:
Program.cs ist für Konfiguration und Middleware-Setup zuständig. Der Seed-Code (Admin-User anlegen) verletzt das Single Responsibility Principle. Ausserdem sind die Zugangsdaten als Magic Strings direkt im Code:

```
// ✗ Aktuell: Magic Strings direkt im Code
admin.PasswordHash = hasher.HashPassword(admin, "Test-1234!");
```

Massnahme:
Seed-Logik in eine eigene statische Klasse DbSeeder.cs extrahieren. Zugangsdaten aus appsettings.Development.json lesen.

```
// ✓ Neu: Eigene Klasse
public static class DbSeeder
{
    public static void Seed(AdresseDbContext db, IPasswordHasher<User> hasher, IConfiguration
config)
```

```

{
    if (db.Users.Any()) return;

    var email    = config["Seed:AdminEmail"]    ?? "admin@example.com";
    var password = config["Seed:AdminPassword"] ?? "Test-1234!";

    var admin = new User { Email = email, DisplayName = "Admin" };
    admin.PasswordHash = hasher.HashPassword(admin, password);
    db.Users.Add(admin);
    db.SaveChanges();
}
}

```

B-02 SRP: DTOs direkt in Controller-Dateien

Dateien: AuthController.cs (Zeile 15–26), SettingsController.cs (Zeile 10–12)

Problem:

RegisterRequest, LoginRequest und SettingsUpdateRequest sind im gleichen File wie der Controller definiert. DTOs sind eigenständige Typen und gehören in eigene Dateien.

```

//  Aktuell: DTO im gleichen File wie Controller
public record RegisterRequest(...);
public record LoginRequest(...);

[Route("auth")]
public class AuthController : ControllerBase { ... }

```

Massnahme:

Eigene Dateien Dtos/AuthDtos.cs und Dtos/SettingsDtos.cs erstellen:

```

backend/AdressverwaltungApi/
  Dtos/
    AuthDtos.cs      ← RegisterRequest, LoginRequest
    SettingsDtos.cs  ← SettingsUpdateRequest

```

B-03 SRP: E-Mail-Benachrichtigung direkt im Controller

Datei: AdressenController.cs, Zeile 112–140

Problem:

SendNewAdresseNotificationAsync ist Notification-Logik — kein Teil der HTTP-Controller-Verantwortung. Bei neuen Benachrichtigungstypen (z.B. SMS, Webhook) müsste der Controller geändert werden.

```

//  Aktuell: Notification-Logik im Controller
private async Task SendNewAdresseNotificationAsync(Adresse adresse) { ... }

```

Massnahme:

Interface INotificationService mit Implementierung EmailNotificationService erstellen:

```
//  Neu: Eigener Service
public interface INotificationService
{
    Task NotifyNewAdresseAsync(Adresse adresse);
}
```

B-04 Fehlerbehandlung: int.Parse ohne Fehlerbehandlung

Datei: EmailService.cs, Zeile 27

Problem:

int.Parse wirft FormatException wenn der Wert in appsettings.json kein gültiger Integer ist. Dies führt zu einem unkontrollierten Absturz.

```
//  Aktuell: Absturz bei ungültigem Wert
var smtpPort = int.Parse(_config["SmtpSettings:Port"] ?? "587");
```

Massnahme:

```
//  Neu: Sicher mit Fallback
if (!int.TryParse(_config["SmtpSettings:Port"], out var smtpPort))
    smtpPort = 587;
```

B-05 Konfiguration: Inkonsistenter Schlüsselname in EmailService

Dateien: EmailService.cs (Zeile 32) vs. docker-compose.yml (Zeile 45)

Problem:

Der Code liest SmtpSettings:SenderEmail, aber docker-compose.yml setzt SmtpSettings__FromAddress. Der Schlüssel stimmt nicht überein — E-Mails werden nie den konfigurierten Absender verwenden.

```
//  Aktuell im Code:
var senderEmail = _config["SmtpSettings:SenderEmail"] ?? smtpUser;

#  Aktuell in docker-compose.yml:
SmtpSettings__FromAddress: noreply@example.com
```

Massnahme:

Einheitlich SmtpSettings:FromAddress verwenden — in Code UND docker-compose:

```
//  Neu:
var senderEmail = _config["SmtpSettings:FromAddress"] ?? smtpUser;
```

B-06 Fehlerbehandlung: String-Vergleich nicht culture-safe

Datei: AuthController.cs, Zeilen 57, 83

Problem:

.ToLower() ist von der System-Locale abhängig (z.B. türkisches i/i-Problem). Für E-Mail-Vergleiche ist StringComparison.OrdinalIgnoreCase korrekt.

```
// ❌ Aktuell:  
u.Email.ToLower() == req.Email.ToLower()
```

Massnahme:

```
// ✅ Neu: Culture-unabhängig  
u.Email.Equals(req.Email, StringComparison.OrdinalIgnoreCase)
```

In EF Core / LINQ:

```
u.Email.ToUpper() == req.Email.ToUpper()  
// oder via EF.Functions.ILike (PostgreSQL)
```

B-07 🟡 Validierung: Fehlende DataAnnotations auf Model-Klassen

Datei: Models/Adresse.cs

Problem:

Validierungsregeln ([Required], [MaxLength]) sind nur im DbContext.OnModelCreating definiert — nicht am Model selbst. Clean Code fordert, dass Constraints dort deklariert werden, wo der Typ definiert ist.

```
// ❌ Aktuell: Keine Annotations am Model  
public string Vorname { get; set; } = string.Empty;
```

Massnahme:

```
// ✅ Neu: Annotations direkt am Model  
[Required]  
[MaxLength(100)]  
public string Vorname { get; set; } = string.Empty;
```

B-08 🟡 DRY: Code-Duplikation zwischen AdressenController und citiesController

Dateien: AdressenController.cs, CitiesController.cs

Problem:

Die CRUD-Operationen GET/POST/PATCH/DELETE sind in beiden Controllern nahezu identisch aufgebaut. Gleiche Muster, gleiche Fehlerbehandlung, gleiche Struktur — verletzt DRY.

Massnahme (für fortgeschrittene Lernende):

Generische Basisklasse ODataCrudController<TEntity> mit gemeinsamer CRUD-Logik:

```
// ✅ Basis-Controller (vereinfacht)  
public abstract class ODataCrudController<TEntity> : ODataController  
    where TEntity : class  
{  
    protected readonly AdresseDbContext _context;
```

```

protected abstract DbSet<TEntity> GetDbSet();

// Get, Post, Patch, Delete gemeinsam implementiert
}

public class AdressenController : ODataCrudController<Adresse> { ... }
public class CitiesController : ODataCrudController<City> { ... }

```

Befunde — Frontend (TypeScript / React)

F-01 🟡 DRY: Doppelte Audit-Feld-Normalisierung in api.ts

Datei: frontend/lib/api.ts, Zeilen 12–45

Problem:

normalizeAdresse und normalizeCity wiederholen die identische Normalisierung der 6 Audit-Felder (createDate, createdBy, changeDate, changedBy, dateFrom, dateTo).

```

// ❌ Aktuell: 6 Zeilen in normalizeAdresse = 6 Zeilen in normalizeCity
createDate: String(raw['createDate'] ?? raw['CreateDate'] ?? ''),
// ... (6x wiederholt)

```

Massnahme:

Hilfsfunktion normalizeAuditFields extrahieren:

```

// ✅ Neu: Gemeinsame Hilfsfunktion
function normalizeAuditFields(raw: Record<string, unknown>) {
  return {
    createDate: String(raw['createDate'] ?? raw['CreateDate'] ?? ''),
    createdBy: String(raw['createdBy'] ?? raw['CreatedBy'] ?? ''),
    changeDate: (raw['changeDate'] ?? raw['ChangeDate'] ?? null) as string | null,
    changedBy: (raw['changedBy'] ?? raw['ChangedBy'] ?? null) as string | null,
    dateFrom: String(raw['dateFrom'] ?? raw['DateFrom'] ?? ''),
    dateTo: (raw['dateTo'] ?? raw['DateTo'] ?? null) as string | null,
  };
}

function normalizeAdresse(raw: Record<string, unknown>): Adresse {
  return {
    id: Number(raw['id'] ?? raw['Id']),
    vorname: String(raw['vorname'] ?? raw['Vorname'] ?? ''),
    // ...
    ...normalizeAuditFields(raw),
  };
}

```

F-02 🟡 UI-Konsistenz: Mixed Controlled/Uncontrolled Inputs in AdresseForm

Datei: frontend/components/AdresseForm.tsx, Zeilen 63–84

Problem:

PLZ und Ort sind Controlled Inputs (value + onChange), alle anderen Felder sind Uncontrolled (defaultValue). Diese Mischung erschwert die Wartung und kann zu schwer findbaren Bugs führen.

```
// ❌ Aktuell: Gemischt
<input name="vorname" defaultValue={...} />    {/* Uncontrolled */}
<input value={plz} onChange={...} />           {/* Controlled */}
```

Massnahme:

Alle Felder als Controlled mit useState implementieren — oder alle als Uncontrolled mit useRef/FormData.

F-03 🔴 Fehlerbehandlung: confirm() und alert() für Benutzerinteraktion

Datei: frontend/app/page.tsx, Zeilen 37, 44

Problem:

confirm() und alert() sind Browser-native Dialoge, die nicht gestaltbar, nicht barrierefrei und in manchen Umgebungen (z.B. iFrames) blockiert sind.

```
// ❌ Aktuell:
if (!confirm('Adresse wirklich löschen?')) return;
alert('Fehler beim Löschen der Adresse.');
```

Massnahme:

Eigene Bestätigungs-Komponente (ConfirmDialog) oder Toast-Notification verwenden:

```
// ✅ Neu: State-basierter Dialog
const [zuLoeschendId, setZuLoeschendId] = useState<number | null>(null);
// Dialog-Komponente rendert bei zuLoeschendId !== null
```

F-04 🟡 TypeScript: Fehlende NextAuth Session-Augmentation

Datei: frontend/lib/auth.ts, Zeile 88

Problem:

Die id-Property auf dem Session-User wird mit einem Type-Cast umgangen, weil NextAuth den Session-Typ nicht kennt.

```
// ❌ Aktuell: Unsicherer Type-Cast
(session.user as { id?: string }).id = token.id as string;
```

Massnahme:

NextAuth Session-Typen via Module Augmentation erweitern (in types/next-auth.d.ts):

```
// ✅ Neu: types/next-auth.d.ts
import 'next-auth';

declare module 'next-auth' {
```

```
interface Session {
  user: {
    id: string;
    name?: string | null;
    email?: string | null;
  };
}
```

F-05 🟡 Namensgebung: Tippfehler sucheLaeuft in AdresseForm

Datei: frontend/components/AdresseForm.tsx, Zeile 27

Problem:

suchelaeuft verstösst gegen camelCase-Konvention (sucheLaeuft). Korrekt wäre:

```
// ❌ Aktuell:
const [suchelaeuft, setSuchelaeuft] = useState(false);

// ✅ Neu:
const [sucheLaeuft, setSucheLaeuft] = useState(false);
```

F-06 🟡 Namensgebung: ladeDaten ohne useCallback in page.tsx

Datei: frontend/app/page.tsx, Zeilen 15–31

Problem:

ladeDaten ist eine innere Funktion ohne useCallback. Sie wird bei jedem Render neu erstellt. Da sie auch im JSX als Click-Handler verwendet wird, ist sie instabil.

```
// ❌ Aktuell:
async function ladeDaten() { ... } // Neue Referenz bei jedem Render
```

Massnahme:

```
// ✅ Neu:
const ladeDaten = useCallback(async () => { ... }, []);
```

Massnahmen-Zusammenfassung

#	Datei	Massnahme	Priorität
B-01	Program.cs	Seed-Logik nach DbSeeder.cs extrahieren	🔴 Hoch
B-02	AuthController.cs / SettingsController.cs	DTOs in Dtos/-Ordner verschieben	🔴 Hoch
B-03	AdressenController.cs	Notification-Logik nach INotificationService	🟡 Mittel

#	Datei	Massnahme	Priorität
B-04	EmailService.cs	int.Parse → int.TryParse	● Hoch
B-05	EmailService.cs + docker-compose.yml	Schlüsselnamen vereinheitlichen	● Hoch
B-06	AuthController.cs	.ToLower() → StringComparison.OrdinalIgnoreCase	● Mittel
B-07	Models/Adresse.cs	[Required] / [MaxLength] hinzufügen	● Mittel
B-08	AdressenController / CitiesController	Generische Basisklasse erwägen	● Mittel
F-01	lib/api.ts	normalizeAuditFields Hilfsfunktion extrahieren	● Mittel
F-02	AdresseForm.tsx	Controlled/Uncontrolled vereinheitlichen	● Mittel
F-03	app/page.tsx	confirm()/alert() → eigene Dialog-Komponente	● Hoch
F-04	lib/auth.ts	NextAuth Session-Augmentation in types/next-auth.d.ts	● Niedrig
F-05	AdresseForm.tsx	suchelaeuft → sucheLaeuft	● Niedrig
F-06	app/page.tsx	ladeDaten mit useCallback	● Niedrig

Positiv hervorgehobene Punkte

Der Code zeigt bereits viele gute Clean-Code-Praktiken:

- **Dependency Injection** durchgehend korrekt (DbContext, IEmailService, ILogger)
- **Interface vor Implementierung:** IEmailService korrekt definiert
- **Sicherheit:** Timing-Angriff-Schutz beim Login, PasswordHash nie in Response
- **Separation of Concerns:** Types (/types), API-Calls (/lib/api.ts), Komponenten (/components) sauber getrennt
- **OData-Verwendung:** AsNoTracking() bei GET korrekt
- **Audit-Felder:** Sauber via AuditableEntity und DbContext.SaveChanges implementiert
- **Kommentare:** Erklären das **Warum**, nicht das **Was**
- **TypeScript:** Strikte Typen, keine any

Nachprüfung vom 4. Oktober 2026 (v2.2.0)

Zweite Durchsicht von Backend und Frontend nach der Umsetzung von B-01 bis B-08 und F-01 bis F-06. Die Befunde B-09 bis B-13 und F-07 bis F-12 sind umgesetzt; Verhalten und Datenbankschema bleiben gleich (51 Backend-Tests grün, erzeugtes Schema vor und nach B-12 identisch).

Backend

#	Prinzip	Befund	Massnahme
B-09	Magic Strings	Konfiguration über Zeichenketten wie <code>config["Jwt:Key"]</code> an mehreren Stellen; <code>double.Parse</code> und <code>bool.Parse</code> ohne Absicherung (wie B-04)	Typisierte Klassen <code>Options/JwtOptions.cs</code> und <code>Options/SmtpOptions.cs</code> (Options-Pattern)
B-10	SRP	<code>AuthController</code> erzeugte das JWT selbst; die E-Mail-Suche stand doppelt in <code>Register</code> und <code>Login</code>	<code>ITokenService / JwtTokenService</code> ; private Methode <code>FindUserByEmailAsync</code>
B-11	DRY, toter Code	<code>AdressenController.Post</code> kopierte die Basismethode, um die Benachrichtigung anzuhängen; ungenutztes Feld <code>_logger</code> ; Not-Found-Meldung dreifach	Erweiterungspunkt <code>OnCreatedAsync</code> in <code>odataCrudController</code> ; Hilfsmethode <code>EntityNotFound</code>
B-12	DRY	Pflichtfelder und Längen standen an den Modellen (B-07) und nochmals in <code>OnModelCreating</code>	Doppelte Fluent-Konfiguration entfernt; es bleiben Tabellennamen und Indizes
B-13	Typsicherheit	<code>SettingsController</code> gab an drei Stellen anonyme Objekte zurück	DTO <code>SettingsResponse</code> ; Zuweisung nur noch an einer Stelle

Dazu: `migration/import_cities.py` entfernt (identische Kopie von `scripts/import_cities.py`), veraltete Kommentare berichtigt.

Frontend

#	Prinzip	Befund	Massnahme
F-07	DRY	Jede Funktion in <code>lib/api.ts</code> wiederholte Aufruf, Statusprüfung und <code>throw</code>	Hilfsfunktionen <code>apiRequest</code> , <code>jsonRequest</code> , <code>leseListe</code> ; Konstanten für die PLZ-Suche statt Zahlen im Code
F-08	DRY	<code>AdresseCreate/CityCreate</code> und <code>AdresseUpdate/CityUpdate</code> mit je gleicher <code>Omit</code> -Liste	Generische Typen <code>CreateOf<T></code> und <code>UpdateOf<T></code> in <code>types/auditable.ts</code>
F-09	DRY	Formular-Klassen, Fehlerbox und Ladeanzeige als gleiches Markup in mehreren Dateien	<code>components/formStyles.ts</code> , <code>Fehlermeldung.tsx</code> , <code>Ladeanzeige.tsx</code>
F-10	Konsistenz (F-03)	<code>DeleteButton</code> (Städte) und «Adresse bearbeiten» nutzten weiterhin <code>confirm()/alert()</code>	<code>ConfirmDialog</code> und Fehler-State; <code>ConfirmDialog</code> mit Prop <code>bestaetigenLabel</code>
F-11	Konsistenz (F-02), Namensgebung	<code>CityForm</code> las Werte über <code>form.elements</code> mit Type-Casts; laden bedeutete in den Formularen «speichert»	Controlled Inputs; Umbenennung in <code>speichert</code>
F-12	Fehler	Der Nav-Link «Adressen» zeigte auf <code>/adressen</code> , eine Seite, die es nicht gibt (die Liste liegt auf <code>/</code>)	Link auf <code>/</code> ; aktiv auf <code>/</code> und unter <code>/adressen/...</code>

Dazu: `package.json` auf Version 2.2.0, veraltete Abhängigkeit `@types/next-auth` entfernt.

Nicht umgesetzt

- Die Adressliste ist eine Client-Komponente, die Städteliste eine Server-Komponente. Eine Vereinheitlichung wäre ein Umbau der Datenbeschaffung.
- Die beiden ausführlichen Fehlerboxen in `app/page.tsx` (mit «Erneut versuchen» bzw. Schliessen-Knopf) bleiben eigenes Markup.
- Die PascalCase-Rückfallebene in den `normalize*`-Funktionen wird vom Backend nicht mehr benötigt, bleibt aber als Absicherung.